Module 6

Working with JSON in Yellowbrick

Parsing · Querying · Flattening · ETL

Section 1

What Is JSON?

Format · Structure · Why It Matters

What Is JSON?

Key-Value Pairs

The basic unit: a name and a value.
"city": "Macau" | "amount": 99.50

Objects { }

A collection of key-value pairs, wrapped in curly braces. Think of it as a row in a flexible table.

Arrays [ ]

An ordered list of values or objects. One transaction can contain many items — all inside a single field.

Nested Structures

Objects can contain other objects or arrays to any depth, representing rich, hierarchical data.

A simple JSON customer record:

```
{
  "customerId":      "CUST-042016",
  "firstName":       "Sarah",
  "lastName":        "Perez",
  "membershipTier":  "Gold",
  "loyaltyPoints":   4200,
  "active":          true,
  "address": {
    "city":    "Macau",
    "country": "SAR"
  },
  "recentCategories": [
    "Books",
    "Electronics",
    "Clothing"
  ]
}
```

Anatomy of a Retail Transaction

Each row in samples.retail.staging holds one complete transaction as a single JSONB value.


```
{
  "transactionId": "TXN-2025-11-01-00000001",
  "timestamp":     "2025-11-01T07:52:55Z",

  "store": {
    "storeId":   "STORE-088",
    "storeName": "Express Retail Store #88",
    "address":   { "city": "Franklin", "state": "PA" }
  },

  "customer": {
    "customerId":     "CUST-042016",
    "firstName":      "Sarah",
    "membershipTier": "Gold"
  },

  "items": [
    { "lineItemId": 1, "name": "BookWorld Books #34245",
      "quantity": 5,   "unitPrice": 773.04,
      "discount": { "type": "percentage", "value": 5 },
      "tax":      { "rate": 0.0875 } }
  ],

  "payment": {
    "method":  "debit_card",
    "details": { "cardType": "Visa", "lastFourDigits": "3723" },
    "status":  "completed"
  },
  "receipt": { "printed": true, "emailed": true, "sms": true }
}
```


- Transaction ID & timestamp
- store object (nested object)
- store.address (object in object)
- customer object
- items — array of objects [ ]
- each item is an object { }
- payment object
- payment.details (nested)
- receipt object

Objects { } and Arrays [ ] — Side by Side

Objects { }

A named container of related key-value pairs.
Like a record — but can be nested inside another record.

```
"customer": {
  "customerId":     "CUST-042016",
  "firstName":      "Sarah",
  "membershipTier": "Gold",
  "address": {
    "city":    "Macau",
    "country": "SAR"
  }
}
```

Tip: Access nested values with dot notation:
data:$.customer.address.city

Arrays [ ]

An ordered list. Each element can be a simple value or a full object. This is how one transaction holds many line items.

```
"items": [
  { "lineItemId": 1,
    "name":      "BookWorld Books",
    "quantity":  5,
    "unitPrice": 773.04 },
  { "lineItemId": 2,
    "name":      "StyleCo Clothing",
    "quantity":  2,
    "unitPrice": 99.99 }
]
```

Note: Arrays can NOT be queried row-by-row with simple SELECT.
Use LATERAL FLATTEN — covered in Section 5.

Why Store JSON in Yellowbrick?

Data arrives as JSON	APIs, mobile apps, and modern systems emit JSON natively. Storing it as-is avoids lossy transformation at ingest time.
Schema flexibility	Not every transaction looks the same — some have promotions, some have discount details, some have mobile payment IDs. JSON handles variable structure gracefully.
Staging before ETL	Land raw JSON in a staging table first, then transform into clean dimensional tables. This is exactly how samples.retail was built — one staging table, one full star schema.
Query without transformation	Yellowbrick lets you query JSON fields directly in SQL. No ETL required just to answer "how many Gold-tier customers paid with Apple Pay last month?"
Mix with relational data	JOIN JSON staging to dimension tables, aggregate across flattened arrays, apply WHERE filters — JSON is a first-class citizen in Yellowbrick SQL.

Section 2

JSON vs JSONB

Two types, one clear choice — and it matters

JSON vs JSONB — Know the Difference

	JSON	JSONB ✓ Recommended
Storage format	Text — stored exactly as written	Binary — pre-parsed and validated on insert
Query performance	Slower — parses text on every query	Faster — structure already decoded
LATERAL FLATTEN	*Not supported*	Fully supported
Whitespace & order	Preserved exactly	Normalized — whitespace stripped, keys ordered
Duplicate keys	Allowed (last value wins)	Deduplicated automatically
Best use case	Exact text archival / audit logs	**Querying, filtering, ETL, analytics**

The Staging Table — Where JSON Lives

Table definition:

```
-- In the samples.retail schema
CREATE TABLE staging (
    data  JSONB        -- one full
);                     -- transaction per row

-- Each row looks like:
SELECT data
FROM   samples.retail.staging
LIMIT  1;

-- Result: one very long JSON value
-- { "transactionId": "TXN-...", ... }
```

One row = one transaction

Every complete JSON transaction document is stored in a single row of the staging table.

Single column: data JSONB

There is only one column. All the structure — store, customer, items, payment — lives inside that one JSONB value.

samples.retail.staging

You already have read access to this table. The cleaned star-schema tables in samples.retail were built FROM this staging table using the ETL patterns covered in Section 6.

Try it: SELECT COUNT(*) FROM samples.retail.staging;

Section 3

Extracting Values from JSON

Accessor syntax · Nested paths · Casting · NULL ON ERROR

The Yellowbrick JSON Accessor Syntax

To read a value from a JSON field in Yellowbrick, use the colon-dollar path syntax:

```
column_name :$. field_name
```

`data:$.transactionId`	`-> "TXN-2025-11-01-00000001"`	*Top-level field — the transaction ID string*
`data:$.customer.membershipTier`	`-> "Gold"`	*Nested field — customer's loyalty tier*
`data:$.summary.grandTotal`	`-> 3993.23`	*Numeric field — total transaction amount*

Tip: The result is still JSON text. Cast it to a SQL type before using it in calculations or comparisons — covered on the next slide.

Navigating Nested Objects — Dot Notation

Chain field names with dots to drill into nested objects. Go as deep as the structure requires.

`data:$.store.storeId`	*store → storeId*
`data:$.store.storeName`	*store → storeName*
`data:$.store.address.city`	*store → address → city*
`data:$.store.address.state`	*store → address → state*
`data:$.customer.firstName`	*customer → firstName*
`data:$.customer.membershipTier`	*customer → membershipTier*
`data:$.cashier.employeeId`	*cashier → employeeId*
`data:$.payment.method`	*payment → method*
`data:$.payment.details.cardType`	*payment → details → cardType*

Key rule: Each dot goes one level deeper into the JSON hierarchy. If the field doesn't exist, Error – Unless NULL ON ERROR specified.

Casting Extracted Values to SQL Types

JSON extraction always returns a JSON fragment — a string. Use :: to cast it to the SQL type you actually need.

```
( data:$.field ) :: TARGET_TYPE
```

JSON Path	Cast To	Example Result	Use For
`data:$.transactionId`	`VARCHAR`	'TXN-2025-11-01-00000001'	*String / text*
`data:$.summary.grandTotal`	`DECIMAL(12,2)`	3993.23	*Currency / math*
`data:$.items[0].quantity`	`INTEGER`	5	*Counting*
`data:$.timestamp`	`TIMESTAMP`	2025-11-01 07:52:55	*Date filtering*
`data:$.receipt.printed`	`BOOLEAN`	true / false	*Flag fields*
`data:$.items[0].tax.rate`	`DECIMAL(5,4)`	0.0875	*Rates / ratios*

Tip: Always wrap the path in parentheses before casting: (data:$.grandTotal)::DECIMAL(12,2) — parens are required.

Accessing Array Elements by Index

Use [0], [1], [2]... to pick a specific item from an array by its position (zero-based).

The items array:

```
"items": [
  { "lineItemId": 1,        ← [0]
    "name":      "Books",
    "quantity":  5 },
  { "lineItemId": 2,        ← [1]
    "name":      "Clothing",
    "quantity":  2 },
  { "lineItemId": 3,        ← [2]
    "name":      "Electronics",
    "quantity":  1 }
]
```

Note: Array indexes are zero-based: the first element is [0], not [1].
Using an index beyond the array length returns NULL — no error.

Querying by index:

```
SELECT
  -- First item's name
  (data:$.items[0].name)::VARCHAR         AS
first_item,

  -- Second item's quantity
  (data:$.items[1].quantity)::INTEGER     AS
second_qty,

  -- First item's unit price
  (data:$.items[0].unitPrice)::DECIMAL(10,2) AS price

FROM samples.retail.staging
LIMIT 5;
```

Tip: Useful for spot-checking, but to get ALL items from ALL transactions use LATERAL FLATTEN — covered in Section 5.

Try it: (data:$.items[0].name)::VARCHAR AS first_item FROM samples.retail.staging LIMIT 3

NULL ON ERROR — Handling Optional Fields Safely

Not every JSON record has the same fields. When a path doesn't exist or a cast fails, Yellowbrick raises an error by default — unless you add NULL ON ERROR.

Without NULL ON ERROR:

```
-- Cash payments have no cardType field.
-- This query FAILS on cash transactions:

SELECT
  (data:$.payment.details.cardType)::VARCHAR
FROM samples.retail.staging;

ERROR: cannot cast NULL or missing
       JSON value to VARCHAR
```

Note: The error happens because cash transactions have "details": {} — the cardType key simply doesn't exist.

Syntax recap: (data:$.field NULL ON ERROR)::TYPE

With NULL ON ERROR:

```
-- Add NULL ON ERROR after the path:

SELECT
  (data:$.payment.details.cardType
     NULL ON ERROR)::VARCHAR  AS card_type,

  (data:$.payment.details.provider
     NULL ON ERROR)::VARCHAR  AS provider,

  (data:$.payment.details.lastFourDigits
     NULL ON ERROR)::VARCHAR  AS last_four

FROM samples.retail.staging
LIMIT 5;
```

Tip: Use NULL ON ERROR wherever a field is optional or varies by record type — discounts, promotions, payment details.

Putting It Together — A Complete SELECT from Staging

Combining everything from this section into one query:

```
SELECT
    -- Transaction identity
    (data:$.transactionId)::VARCHAR                              AS transaction_id,
    (data:$.timestamp)::TIMESTAMP                                AS txn_time,

    -- Store
    (data:$.store.storeId)::VARCHAR                              AS store_id,
    (data:$.store.storeName)::VARCHAR                            AS store_name,
    (data:$.store.address.city)::VARCHAR                         AS city,
    (data:$.store.address.state)::VARCHAR                        AS state,

    -- Customer
    (data:$.customer.firstName)::VARCHAR                         AS first_name,
    (data:$.customer.lastName)::VARCHAR                          AS last_name,
    (data:$.customer.membershipTier)::VARCHAR                    AS tier,

    -- Payment (optional fields use NULL ON ERROR)
    (data:$.payment.method)::VARCHAR                             AS pay_method,
    (data:$.payment.details.cardType    NULL ON ERROR)::VARCHAR AS card_type,
    (data:$.payment.details.provider   NULL ON ERROR)::VARCHAR  AS provider,

    -- Transaction totals
    (data:$.summary.subtotal)::DECIMAL(12,2)                     AS subtotal,
    (data:$.summary.grandTotal)::DECIMAL(12,2)                   AS grand_total,
    (data:$.receipt.emailed)::BOOLEAN                            AS emailed

FROM samples.retail.staging
LIMIT 10;
```

What the Results Look Like

The query from the previous slide returns a clean relational result set — familiar rows and columns:

txn_time	store_id	first_name	tier	pay_method	card_type	grand_total	emailed
2025-11-01 07:52	STORE-088	Sarah	Gold	debit_card	Visa	3,993.23	true
2025-12-26 10:03	STORE-100	Charles	Bronze	cash	—	11,395.19	false
2025-11-11 05:06	STORE-008	Robert	Bronze	debit_card	Mastercard	17,354.29	true
2026-03-26 01:15	STORE-024	Nancy	Platinum	mobile_payment	—	20,744.85	false
2025-06-13 05:45	STORE-069	Patricia	Bronze	debit_card	Visa	12,696.89	false

✓ JSON is gone — the result is plain SQL rows and columns, ready to join, aggregate, or export.

✓ NULL ON ERROR fields (card_type, provider) show null naturally for cash/mobile — no errors.

✓ Dates, decimals, and booleans are proper SQL types — you can use them in WHERE, GROUP BY, and ORDER BY.

Section 4

Filtering JSON Data

WHERE clauses · Dates · NULL handling

Filtering with WHERE — JSON Fields Work Like Any Column

Once extracted and cast, JSON values behave exactly like regular SQL columns in WHERE clauses.

```
-- Filter by membership tier
SELECT (data:$.customer.firstName)::VARCHAR       AS name,
       (data:$.summary.grandTotal)::DECIMAL(12,2) AS total
FROM   samples.retail.staging
WHERE  (data:$.customer.membershipTier)::VARCHAR = 'Gold';

-- Filter by payment method
SELECT (data:$.transactionId)::VARCHAR            AS txn_id,
       (data:$.summary.grandTotal)::DECIMAL(12,2) AS total
FROM   samples.retail.staging
WHERE  (data:$.payment.method)::VARCHAR = 'mobile_payment';

-- Filter by Apple Pay (nested optional field)
SELECT (data:$.customer.firstName)::VARCHAR       AS name,
       (data:$.summary.grandTotal)::DECIMAL(12,2) AS total
FROM   samples.retail.staging
WHERE  (data:$.payment.details.provider NULL ON ERROR)::VARCHAR = 'Apple Pay';

-- Combine filters
SELECT (data:$.transactionId)::VARCHAR            AS txn_id,
       (data:$.summary.grandTotal)::DECIMAL(12,2) AS total
FROM   samples.retail.staging
WHERE  (data:$.customer.membershipTier)::VARCHAR = 'Platinum'
  AND  (data:$.summary.grandTotal)::DECIMAL(12,2) > 15000;
```

Tip: Always cast the field before comparing — don't compare raw JSON text to a number or date.

NULL Handling — When Fields Don't Exist

JSON documents in the same table don't have to have all the same fields. Here's how Yellowbrick handles the gaps.

Field is present and has a value

```
Data: "discount": {"type": "percentage", "value":
5}
SQL: (data:$.items[0].discount.type)::VARCHAR
```

→ 'percentage'

Field is present but value is null

```
Data: "discount": {"type": null, "value": 0}
SQL: (data:$.items[0].discount.type)::VARCHAR
```

→ NULL — the field exists but is null

Field is completely absent

```
Data: "payment": {"method": "cash", "details": {}}
SQL: (data:$.payment.details.cardType)::VARCHAR
```

→ ERROR without NULL ON ERROR

Fix: use NULL ON ERROR

```
Data: "payment": {"method": "cash", "details": {}}
SQL: (data:$.payment.details.cardType NULL ON ERROR)::VARCHAR
```

→ NULL — safely returns null

Section 5

Flattening Arrays with LATERAL FLATTEN

The key to querying one-to-many JSON relationships

The Challenge — One Row, Many Line Items

Each staging row holds one transaction. That transaction can have 1–10 line items inside the items array. How do you get one result row per item?

staging (1 row)

```
data = {
  "transactionId": "TXN-001",
  "customer": {...},
  "items": [
    { item 1: Books, qty 5 },
    { item 2: Clothing, qty 2 },
    { item 3: Shoes, qty 1 },
    { item 4: Electronics, qty 3 }
  ]
}
```

LATERAL FLATTEN →

Result (4 rows)

txn_id	item_name	qty
TXN-001	**Books**	5
TXN-001	**Clothing**	2
TXN-001	**Shoes**	1
TXN-001	**Electronics**	3

Key insight: Transaction fields (customer, store, payment) repeat on every output row — one copy per item. This is how you JOIN array elements back to the parent transaction.

LATERAL FLATTEN — The Syntax

```
FROM table_alias,  LATERAL FLATTEN(
table_alias.column:$.array_field )  AS alias
```

Applied to the retail staging table:

```
SELECT
    (st.data:$.transactionId)::VARCHAR          AS transaction_id,
    (st.data:$.customer.membershipTier)::VARCHAR AS tier,

    -- Fields from the flattened item
    ((item.item):$.value.name)::VARCHAR          AS product_name,
    ((item.item):$.value.category)::VARCHAR      AS category,
    ((item.item):$.value.quantity)::INTEGER      AS qty,
    ((item.item):$.value.unitPrice)::DECIMAL(10,2) AS unit_price

FROM samples.retail.staging AS st,
     LATERAL FLATTEN(st.data:$.items) AS item

LIMIT 15;
```

st = alias for the staging row (top-level fields). **item** = alias for the flatten result — access via **`(item.item):$.value.fieldname`**

Understanding the Flatten Result — item.item and :$.value

The alias returned by LATERAL FLATTEN has a specific structure. Understanding it prevents the most common beginner mistake.

The alias (item)

The alias you gave the LATERAL FLATTEN result.

In our examples: AS item

This is just a name — you can call it anything.

```
LATERAL FLATTEN(
  st.data:$.items
) AS item
```

item.item

The internal column inside the flatten result. The alias has a column also called "item" — so you write alias.item.

This is the full JSON object for one array element.

```
item.item
-> {"lineItemId":1,
    "name":"Books",
    "quantity":5,...}
```

:$.value.field

Use :$.value to navigate inside the item, then add the field name.

:$.value is the key — it points to the object content of this array element.

```
((item.item):$.value
  .quantity)::INTEGER
```

Full pattern: ((item.item):$.value.fieldName)::TYPE — double parens wrapping the whole thing, then cast.

Full LATERAL FLATTEN Example — All Line Items

One query to get every product sold in every transaction — with customer and payment context:

```
SELECT
    -- Transaction context
    (st.data:$.transactionId)::VARCHAR                          AS txn_id,
    (st.data:$.customer.firstName)::VARCHAR                     AS customer,
    (st.data:$.customer.membershipTier)::VARCHAR                AS tier,
    (st.data:$.payment.method)::VARCHAR                         AS pay_method,

    -- Line item details (from the flattened array)
    ((item.item):$.value.lineItemId)::INTEGER                   AS line_num,
    ((item.item):$.value.name)::VARCHAR                         AS product_name,
    ((item.item):$.value.category)::VARCHAR                     AS category,
    ((item.item):$.value.brand)::VARCHAR                        AS brand,
    ((item.item):$.value.quantity)::INTEGER                     AS qty,
    ((item.item):$.value.unitPrice)::DECIMAL(10,2)              AS unit_price,
    ((item.item):$.value.discount.type    NULL ON ERROR)::VARCHAR AS discount_type,
    ((item.item):$.value.discount.amount  NULL ON ERROR)::DECIMAL(10,2) AS discount_amt,
    ((item.item):$.value.total)::DECIMAL(12,2)                  AS line_total

FROM   samples.retail.staging     AS st,
       LATERAL FLATTEN(st.data:$.items) AS item

ORDER  BY st.data:$.transactionId, ((item.item):$.value.lineItemId)::INTEGER
LIMIT  20;
```

Tip: NULL ON ERROR on discount fields prevents errors on items sold at full price — not every item has a discount.

FLATTEN + FILTER — Combining Everything

Find all discounted items purchased by Gold-tier customers — one query against the staging table:

```
SELECT
    (st.data:$.customer.firstName)::VARCHAR                    AS customer,
    (st.data:$.customer.membershipTier)::VARCHAR               AS tier,
    (st.data:$.store.address.city)::VARCHAR                    AS city,
    (st.data:$.payment.method)::VARCHAR                        AS pay_method,

    ((item.item):$.value.name)::VARCHAR                        AS product,
    ((item.item):$.value.category)::VARCHAR                    AS category,
    ((item.item):$.value.quantity)::INTEGER                    AS qty,
    ((item.item):$.value.unitPrice)::DECIMAL(10,2)             AS unit_price,
    ((item.item):$.value.discount.type   NULL ON ERROR)::VARCHAR AS disc_type,
    ((item.item):$.value.discount.value  NULL ON ERROR)::DECIMAL AS disc_pct,
    ((item.item):$.value.discount.amount NULL ON ERROR)::DECIMAL(10,2) AS disc_amt,
    ((item.item):$.value.total)::DECIMAL(12,2)                 AS line_total

FROM   samples.retail.staging AS st,
       LATERAL FLATTEN(st.data:$.items) AS item

WHERE  (st.data:$.customer.membershipTier)::VARCHAR = 'Gold'
  AND  ((item.item):$.value.discount.amount NULL ON ERROR)::DECIMAL(10,2) > 0

ORDER  BY disc_amt DESC;
```

Key point: Filters the parent row (Gold tier) AND within flattened items (discount > 0) — all in one SQL statement.

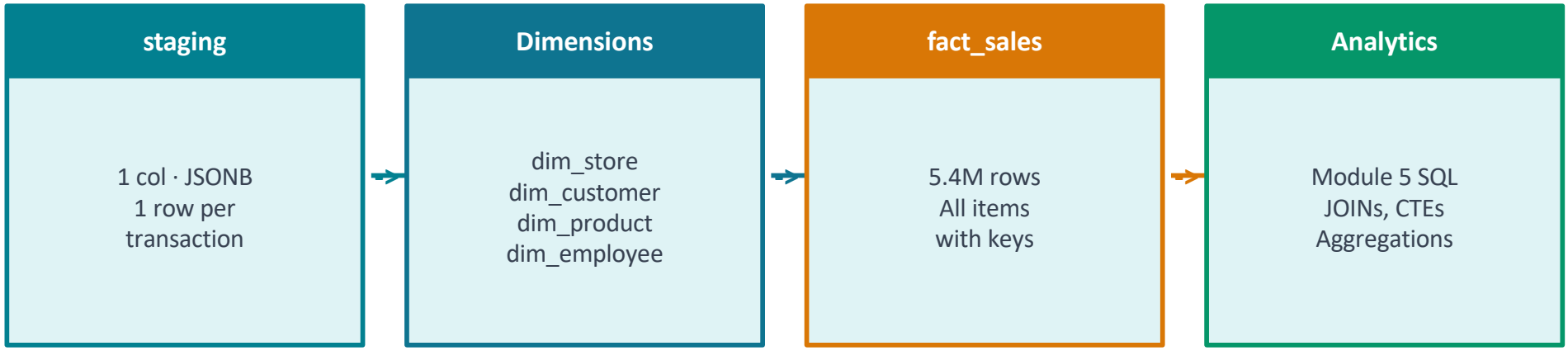
Section 6

From JSON to Structured Tables

The ETL pattern that built samples.retail

The Big Picture — JSON Staging to Star Schema

The samples.retail schema you used in Module 5 was built entirely from the staging table you've been querying. Here's the flow:



1	Extract stores, customers, products, employees → populate dimension tables via INSERT...SELECT DISTINCT from staging
2	LATERAL FLATTEN items array → join to dimension keys → INSERT into fact_sales (one row per line item)
3	Query fact_sales with familiar SQL — all JSON is gone, everything is typed relational columns

ETL Step 1 — Extracting Dimension Tables

Each SELECT DISTINCT pulls unique entities from the JSON. The WHERE NOT EXISTS prevents duplicate inserts on re-runs.

```
-- Populate dim_store from JSON
INSERT INTO dim_store (store_id, store_name, city, state, postal_code, country)
SELECT DISTINCT
    (data:$.store.storeId)::VARCHAR            AS store_id,
    (data:$.store.storeName)::VARCHAR          AS store_name,
    (data:$.store.address.city)::VARCHAR       AS city,
    (data:$.store.address.state)::VARCHAR      AS state,
    (data:$.store.address.postalCode)::VARCHAR AS postal_code,
    (data:$.store.address.country)::VARCHAR    AS country
FROM staging st
WHERE NOT EXISTS (
    SELECT 1 FROM dim_store s
    WHERE s.store_id = (st.data:$.store.storeId)::VARCHAR
);

-- Populate dim_customer from JSON  (same pattern)
INSERT INTO dim_customer (customer_id, first_name, last_name, membership_tier)
SELECT DISTINCT
    (data:$.customer.customerId)::VARCHAR     AS customer_id,
    (data:$.customer.firstName)::VARCHAR      AS first_name,
    (data:$.customer.lastName)::VARCHAR       AS last_name,
    (data:$.customer.membershipTier)::VARCHAR AS membership_tier
FROM staging st
WHERE NOT EXISTS (
    SELECT 1 FROM dim_customer c
    WHERE c.customer_id = (st.data:$.customer.customerId)::VARCHAR
);
```

Same pattern applies to dim_employee and dim_product — SELECT DISTINCT the relevant JSON fields, cast, insert with deduplication check.

ETL Step 2 — Flattening Items into fact_sales

LATERAL FLATTEN turns each item in the items array into its own row in the fact table:

```
INSERT INTO fact_sales (
    store_key, customer_key, employee_key, product_key,
    transaction_id, line_item_id, quantity, unit_price,
    discount_type, discount_amount, tax_rate, total,
    transaction_timestamp
)
SELECT
    s.store_key,
    c.customer_key,
    e.employee_key,
    p.product_key,

    (st.data:$.transactionId)::VARCHAR                          AS transaction_id,
    ((item.item):$.value.lineItemId)::INTEGER                   AS line_item_id,
    ((item.item):$.value.quantity)::INTEGER                     AS quantity,
    ((item.item):$.value.unitPrice)::DECIMAL(10,2)              AS unit_price,
    ((item.item):$.value.discount.type   NULL ON ERROR)::VARCHAR AS discount_type,
    ((item.item):$.value.discount.amount NULL ON ERROR)::DECIMAL(10,2) AS discount_amount,
    ((item.item):$.value.tax.rate)::DECIMAL(5,4)                AS tax_rate,
    ((item.item):$.value.total)::DECIMAL(12,2)                  AS total,
    TO_TIMESTAMP(REPLACE(REPLACE(
        (st.data:$.timestamp)::VARCHAR,'T',' '),'Z',''),
        'YYYY-MM-DD HH24:MI:SS.US')                             AS transaction_timestamp

FROM staging st,
     LATERAL FLATTEN(st.data:$.items) AS item
JOIN dim_store    s ON s.store_id    = (st.data:$.store.storeId)::VARCHAR
JOIN dim_customer c ON c.customer_id = (st.data:$.customer.customerId)::VARCHAR
JOIN dim_employee e ON e.employee_id = (st.data:$.cashier.employeeId)::VARCHAR
JOIN dim_product  p ON p.product_id  =
     ((item.item):$.value.productId)::VARCHAR;
```

Tip: The JOINs to dimension tables swap JSON strings for integer surrogate keys — result is a pure relational fact table.

Before vs After — Querying Staging vs fact_sales

Both approaches answer the same question. The structured table is faster and cleaner for analysts.

Querying staging directly

```
SELECT
  (st.data:$.customer.membershipTier)
      ::VARCHAR                AS tier,
  ((item.item):$.value.category)
      ::VARCHAR                AS category,
  SUM(((item.item):$.value.total)
      ::DECIMAL(12,2))         AS revenue

FROM samples.retail.staging AS st,
     LATERAL FLATTEN(
       st.data:$.items) AS item

GROUP BY 1, 2
ORDER BY revenue DESC;
```

Querying fact_sales (post-ETL)

```
SELECT
  c.membership_tier          AS tier,
  p.category                 AS category,
  SUM(f.total)               AS revenue

FROM   fact_sales f
JOIN   dim_customer c
       ON f.customer_key = c.customer_key
JOIN   dim_product  p
       ON f.product_key  = p.product_key

GROUP  BY 1, 2
ORDER  BY revenue DESC;
```

Same result, different paths: Both return identical output. fact_sales is for day-to-day analytics; staging queries are for exploration and new ETL development.

Section 7

Summary & Cheat Sheet

Key syntax · Best practices · What's next

JSON Syntax Cheat Sheet

Basic Extraction

```
(data:$.fieldName)::TYPE
(data:$.obj.subField)::TYPE
(data:$.array[0].field)::TYPE
```

Optional Fields

```
(data:$.field  NULL ON ERROR)::TYPE
-- Returns NULL instead of error
-- Use for any field that may be
absent
```

LATERAL FLATTEN

```
FROM tbl AS t,
     LATERAL FLATTEN(t.col:$.array)
AS a
((a.a):$.value.field)::TYPE
```

Useful Casts

```
::VARCHAR    ::INTEGER
::DECIMAL(p,s)
::TIMESTAMP  ::BOOLEAN   ::DATE
::DECIMAL(5,4)  for rates/ratios
```

Filtering

```
WHERE (data:$.tier)::VARCHAR =
'Gold'
AND   (data:$.total)::DECIMAL >
10000
AND   (data:$.field NULL ON ERROR)
IS NOT NULL
```

NULL ON ERROR on FLATTEN

```
LATERAL FLATTEN(
  t.col:$.array NULL ON ERROR
) AS a  -- skip missing arrays
```

Best Practices for Working with JSON in Yellowbrick

✓ **Always cast extracted values**	JSON extractions return raw text. Always cast to the right SQL type (::INTEGER, ::DECIMAL, ::TIMESTAMP) before using in math, comparisons, or ORDER BY.
✓ **Use JSONB, not JSON**	JSONB is binary-optimized — faster queries, supports FLATTEN, normalizes whitespace. Use JSON only when you need exact text preservation (audit logs, archival).
✓ **Use NULL ON ERROR on optional fields**	Any JSON field that isn't guaranteed on every row needs NULL ON ERROR. Payment details, discount info, promotions — these vary by transaction.
✓ **ETL to relational tables for production reporting**	Querying staging is great for exploration and development. For dashboards and regular reporting, ETL the data into fact/dimension tables — queries will be faster and SQL will be simpler.
✓ **Use LATERAL FLATTEN for array analysis — not index access**	Array index access (items[0]) is brittle and only sees one element. LATERAL FLATTEN processes every element across every row — use it for any real analysis.